Phần 1: Mã nguồn câu lệnh Windows PowerShell

Các câu lệnh được thực thi tuần tự từ thư mục làm việc hiện tại:

PowerShell

```
# 1. Tạo cây thư mục với tham số -Force để tự động tạo tất cả thư mục cha nếu chưa tồn tại
New-Item -ItemType Directory -Path ".\Farm-Manager\code\app", ".\Farm-Manager\database\db", ".\Farm-Manager\logs\guides" -Force

# 2. Tạo 3 file rỗng bên trong thư mục app
New-Item -ItemType File -Path ".\Farm-Manager\code\app\main.py", ".\Farm-Manager\code\app\models.py", ".\Farm-Manager\code\app\utils.py"

# 3. Sao chép đệ quy toàn bộ thư mục logs (bao gồm cả thư mục con 'guides' và file bên trong) sang 'logs-backup'
Copy-Item -Path ".\Farm-Manager\logs" -Destination ".\Farm-Manager\logs-backup" -Recurse
```

Giải thích các câu lệnh và tham số:

- `New-Item`: Cmdlet dùng để tạo mục mới (thư mục hoặc tập tin) trên hệ thống tệp.
- `-ItemType Directory / File`: Chỉ định rõ loại mục cần tạo là thư mục (`Directory`) hay tập tin (`File`).
- `-Force`: Cho phép tạo thư mục lồng nhau nhiều cấp trong một lần chạy mà không báo lỗi nếu thư mục cha chưa có sẵn; đồng thời ghi đè/bỏ qua nếu đối tượng đã tồn tại.
- `Copy-Item`: Cmdlet dùng để sao chép dữ liệu từ vị trí nguồn (`-Path`) đến vị trí đích (`-Destination`).
- `-Recurse`: Bắt buộc sao chép đệ quy, tức là quét và sao chép toàn bộ thư mục con (ở đây là thư mục `guides`) cùng mọi dữ liệu bên trong thư mục `logs` thay vì chỉ sao chép phần vỏ thư mục rỗng bên ngoài.

Phần 2: Kiểm thử

1. Câu lệnh kiểm tra cấu trúc

Sau khi chạy xong các lệnh trên, kiểm tra cây thư mục vừa tạo bằng lệnh:

PowerShell
```
# Liệt kê danh sách các thư mục cấp 1 bên trong Farm-Manager
Get-ChildItem -Path .\Farm-Manager
# Hoặc sử dụng bí danh (alias):
ls .\Farm-Manager
```

Để xem trực quan toàn bộ cấu trúc cây thư mục và file bên trong:

PowerShell
```
Get-ChildItem -Path .\Farm-Manager -Recurse
# Hoặc lệnh hiển thị dạng sơ đồ cây:
tree .\Farm-Manager /F
```

Kết quả hiển thị trong `.\Farm-Manager` sẽ gồm 4 thư mục ngang hàng:

- `code` (chứa app, bên trong có `main.py`, `models.py`, `utils.py`)
- `database` (chứa db)
- `logs` (chứa `guides`)
- `logs-backup` (chứa bản sao nguyên vẹn của `logs`, gồm cả `guides`)

2. Giải thích: Tại sao cần dùng đường dẫn tương đối (`.\`) thay vì tuyệt đối (`C:\...`)?

- **Tính di động (Portability):** Đường dẫn tương đối giúp script chạy được ở bất kỳ đâu (ổ đĩa `D:\`, `E:\`, trong thư mục dự án của người dùng khác, hoặc máy chủ ảo Cloud Server) mà không lo bị lỗi `PathNotFound`.
- **Tránh rủi ro gắn cứng (Hard-coding):** Nếu dùng `C:\...`, script chỉ hoạt động trên một máy duy nhất có đúng cấu trúc tên người dùng và ổ đĩa đó; khi đưa lên môi trường Server hoặc chuyển cho đồng đội sẽ bị gãy toàn bộ đường dẫn.
- **An toàn môi trường làm việc:** Khi thao tác bằng `.\`, toàn bộ tác vụ tạo/sao chép chỉ diễn ra gói gọn trong phạm vi thư mục dự án hiện tại, tránh vô tình tạo nhầm hoặc can thiệp vào các thư mục hệ thống bên ngoài ổ `C:`.